Handoff 24: Python unit 2, part 1 (EDU-04: arithmetic, int and float, converting input)

Date: 2026-10-27 (America/Toronto)
Written by: Claude Code (implementer), in a separate git worktree
For: the human maintainer, and ChatGPT or Claude chat (reviewer) without repository access
Status: **in_review**: one pull request (#PR), not merged, not deployed. **No database migration, no new dependency, no new API field.** Unit 1 content is untouched.

> **How to use this file:** upload it with `build.md`, `docs/plan/teaching-design.md`, `docs/plan/lesson-feedback.md`, `docs/plan/browser-checks.md` and handoffs 19 and 22. The pull request diff is the source of truth (`build.md` section 14). The appendix has every card, hint and check case word for word.

Starter prompt for the chat assistant

```
You are reviewing Python Academy stage 24 (EDU-04 part 1): the first
three lessons of Python unit 2 "Numbers and decisions" (py.arithmetic,
py.int-float, py.convert-input), for a grade 9 beginner who finished
unit 1. Each challenge is a browser check: the learner's code runs in
their browser against visible and hidden test cases, and the API only
compares what it printed. I'm attaching build.md, teaching-design.md,
lesson-feedback.md, browser-checks.md and this handoff. Read the
appendix: check that the wording is calm, original and clear for a
grade 9 reader, one idea per card; that the checks measure the skill
and accept other correct solutions; that no hint, tagged hint or
similar example gives the answer; and that the cards are accurate
Python. Explain at a first-year computer engineering level.
```

1. Step 0 (checks before building)

Check	Result
`git fetch --all`, `gh pr list --state all`	Done. One open PR: #28 (README-01), not related; this PR is not stacked on it
PR #27 (UI-05) merged into `main`?	**Yes**, 2026-10-27 16:57 UTC (`f4d11e9`)

Check	Result
Unit 2 content on any branch?	**None**: no `content/py/u02`, and no file or path naming `py.arithmetic`, `int-float` or `convert-input` on any local or remote branch
Where the work happened	A separate worktree, `../python-academy-u02`, on a new branch `content/py-u02-part1` from `origin/main` at `f4d11e9`. The main folder and other worktrees were not switched

2. Handoff record (build.md template)

Field	Value
Task	EDU-04 part 1: the first three of unit 2's six lessons, with browser checks, rules, tagged hints, a hint ladder, similar examples and other ways; release them in the catalog
Human goal	A learner who finished unit 1 can start unit 2 live and learn arithmetic, int and float, and turning replies into numbers, with checks they can trust
Repository and branch	`torontojohn/python-academy`, branch `content/py-u02-part1`
Commits	`cfa0118`: lessons, release, tests (pushed early when a usage limit hit). Then the explanation fix, docs and this handoff
Implementer / reviewer	Claude Code / the owner and a chat assistant
Non-goals	Lessons 4 to 6 (`py.compare`, `py.boolean-logic`, `py.elif`: the proposed next PR), a unit project, a unit check, construct rule IDs, any unit 1 change, `backlog.md` and `decision-log.md`
Data and compatibility	No migration. New lesson, task and badge IDs only; nothing reused. XP and badges use the existing ledger (100 XP and one badge per lesson challenge, awarded once)
Deployment status	Not requested

Files changed

File	Why
`backend/content/py/u02/py.arithmetic.json`, `py.int-float.json`, `py.convert-input.json` (new)	The three lessons (appendix)
`backend/content/catalog.json`	`py.u02` released with the three lessons; its summary now says the rest is coming

File	Why
`frontend/app/adventure.tsx`	One sentence: the end of a unit other than unit 1 no longer says "Unit complete" (section 3)
`frontend/app/feedback.ts`, `frontend/tests/fixtures/translations.snapshot.json`, `docs/plan/lesson-feedback.md`	The unit 2 explanation for joining text and a number with `+` now also says how to **add**: with `int(...)` (section 5)
`backend/tests/case_runner.py` (new, tests only)	Runs authored programs once per check case, the way the browser worker does, so tests can use the real judge
`backend/tests/test_unit2_lessons.py` (new, 83 tests)	Section 6
`frontend/tests/unit2.spec.ts` (new, 25 tests), `frontend/tests/fixtures/unit2-api.json` (new)	Browser tests with real Pyodide; the recorded real API answers they use
`backend/tests/fixtures/catalog_v1.json`, `frontend/tests/fixtures/lesson-units-api.json`	The catalog now has unit 2 released (the diff shows only that)
`backend/tests/test_academy.py`, `test_browser_checks.py`, `test_content.py`, `test_lesson_units.py`, `test_recommend.py`, `frontend/tests/academy.spec.ts`	Expectations that said unit 2 was coming soon, or counted 5 lessons in the course
`backend/content/README.md`, `docs/plan/browser-checks.md`	Unit 2 now uses browser checks
`docs/handoffs/2026-10-27-24-unit2-part1.md` and `.pdf`	This handoff

3. How it works

The story. The adventure gains a score and riddles with number answers. Lesson 1 shares a chest of coins between friends; lesson 2 averages two riddle scores (one can be negative); lesson 3 asks the player for a score and a riddle's points and prints the score if the riddle is solved and if it is missed.

Each lesson has unit 1's five cards: discover, a worked example (with comments naming each step), predict, "Try it", and the challenge. Predict-then-run: card 3 asks for a prediction about the example, and card 4 asks the learner to predict, then run and compare. Worked examples use a different context from the challenge (a gem score, a walk to the market, a bridge toll).

Support fades: lessons 1 and 2 give the two `int(input(...))` lines in the starter ("lesson 3 explains how"); lesson 3's starter is three comments only, and the learner writes the conversions.

Checks (ENG-03, unchanged): each challenge has 5 cases, 3 visible and 2 hidden, with the 3-second limit. The program prints **just the numbers**, so the check measures the arithmetic, not the wording. Edge cases taught: an exact share (leftover 0), fewer coins than friends (share 0), `/` giving a float even for a whole answer (`7.0`), a negative average and `int()` cutting toward zero (`-0.5` → `0`), a score going below zero, and an **empty** reply. Hidden cases use other numbers, so hard-coded output fails.

Rules and messages. Browser checks name a wrong-output result `<lesson>.case.<case id>` (handoff 22), for example `py.arithmetic.case.fewer`. Its message is the judge's sentence, built from the case's description: "For 3 coins and 5 friends, line 1 should be "0", but your program printed "1"." Hidden cases are named only by their description ("a big chest (a hidden test case)"). Browser-check challenges cannot have content `messages` (the API refuses them), so the per-case teaching words are in **tagged hints**, offered first when that case fails.

The end of what is released. After lesson 3, the lesson menu shows 3 of 3, the card's button is "Revisit this quest" (there is no next lesson), and the box under the card now says: **"You finished every lesson released so far. That is all of Numbers and decisions for now. See the course for what comes next."** It used to say "Unit complete. Every lesson in … is done", which is not true while unit 2 has three of its six lessons. Unit 1's own ending is unchanged.

4. Decisions and why

Decision	Options considered	Reason	Who decided
Release unit 2 with three lessons	Keep it `coming_soon` until part 2	Your preference, and the pages end honestly: the course page already says "You have finished every available lesson", the unit summary says what comes next, and the lesson page's ending was reworded. The one place that is not quite right is the home card (section 7)	Owner preference; implementer checked the pages
Lessons 1 and 2 hand over the `int(input())` lines	No input in lessons 1 and 2	Without replies, every case would have the same input and printing the expected text would pass. With replies, hidden cases catch it. The starter says lesson 3 explains it	Implementer (**assumption 1**)
"Print just the two numbers"	Accept labels or sentences	The judge compares lines exactly; numbers only means the check judges the maths, never wording, and any correct method passes (`divmod`, `int(c / f)`, `* 0.5`, `or "0"`)	Implementer

Decision	Options considered	Reason	Who decided
Empty reply is a test case; a reply like `ten` is not	A `ten` case	A correct program stops with a ValueError on `ten`, and the contract has no "expected error" case (that is a unit 6 checker). The empty reply can be handled with `if reply == "":`, which unit 1 taught. `ten` is covered on the predict card and the "Try it" card instead	Implementer (**assumption 2**)
Fix-the-bug as closely as five cards allow	A new task type	A fix-the-bug card is a schema change (teaching-design section 3). Lesson 3's predict card asks what goes wrong with `ten`, and its "Try it" card has the learner delete `int(`, run, read the explanation, and repair it	Implementer (**assumption 3**)
One practice badge per lesson	No per-lesson badge, as `course-catalog.md` section 6.2 proposes	Your instruction ("100 XP and one badge per lesson challenge"). The plan's line should change in your docs PR	Owner
No `require`/`forbid` constructs, so no construct rule IDs needed	Require `int()` in lesson 3	The cases already force the behaviour: without `int()`, Python stops or prints `13.0`. So no rule naming was invented (section 9 lists what would be needed)	Implementer
Reword one unit 2 error explanation	Leave it	"Text and numbers cannot be joined with `+`" told learners to use `str()`. In lesson 3 the right fix is usually `int()` on the reply, so it now names both: add with `int(...)`, join with `str(...)`. Unit 1 never shows this explanation	Implementer
A test-only stand-in for the browser worker (`case_runner.py`)	Only browser tests	The judge needs per-case reports; producing them in CPython 3.12 (the same Python version as Pyodide 0.27) lets 80+ programs and thousands of hint combinations be checked in seconds. The browser tests then prove Pyodide sends the same reports	Implementer
Catalog and course versions unchanged	Bump them	Nothing saved refers to them; lesson, task and badge records are new at version 1	Implementer

5. Bugs found and how they were fixed

Bug or near-miss	How it was found	Fix
"Unit complete" after three of six lessons	Checking what the page does after the last available lesson	Reworded (section 3); guarded by the "after the last released lesson" browser test
Misleading advice for lesson 3. Adding a number to a reply that was still text gave "Turn the number into text with `str(...)`", which makes `10 + 3` print `103`	The no-mock run, reading each explanation	The explanation now says how to add, with `int(...)`; the snapshot test shows the change; the int-float browser test checks the new words
Near-miss: my first hint audit read prose as code (`print(coins). Joining` parses as Python) and so missed real code in the hints	A test that the audit finds the code it should	The audit only takes assignments, calls and `if` headers; `test_the_audit_finds_the_code_in_the_hints` guards it
Near-miss: 9 "CSP violations" in my scratch run	Counting console messages	They came from my own axe script (an inline script without the page's nonce), one per scan; the app logged none, and the browser tests assert no violations
Near-miss: 13 browser tests failed in one full run	The run	The run shared the machine with the backend suites and took 32 minutes; alone, all 150 pass in 1.5 minutes. One unit 1 test ("checker service errors…") also failed once that way earlier and then passed 8 of 8

6. How it was tested

Commands and results (in the worktree, macOS, Python 3.12.14, Node 24)

Command	Result
`cd backend && DATABASE_URL=sqlite:///<temp>.db ../../python-academy/backend/.venv/bin/python -m pytest -q`	**451 passed, 11 skipped** (368 before, plus 83 new)
Same with `TEST_POSTGRES_URL=<local PostgreSQL 16 from pgserver>`	**462 passed**
`cd frontend && npx tsc --noEmit && npm run build`	OK
`cd frontend && CI=1 npx playwright test`	**150 passed** (125 before, plus 25 new)

Command	Result
No-mock run: real API (uvicorn), production build, real Pyodide, 8 programs per lesson, plus axe-core 4.10.3 (WCAG 2.0/2.1/2.2 A and AA, best practices) on every card of every lesson, the results, the course page, the catalog and the ending, at 1280 px and 375 px	Every result as expected; **0 axe violations in 47 scans**; no sideways scrolling

What the tests cover

Required test	Where
Content validation for every new lesson	`test_each_lesson_is_a_browser_check_with_visible_and_hidden_cases`, unit release, badges, fading; the app's own load-time validation
What the cards say is true	Every worked example's output, every number claimed on a card (13 claims), the predict answers, run in Python
Correct programs pass	The reference, 3 creative solutions and the 2 "other ways" per lesson
Wrong programs fail with the right rule	31 samples: every case rule (hidden cases too) has one, and each error type a tagged hint names. Hard-coding one answer, or answering only the visible cases, fails (a hidden case catches the second)
Hint audit	Code in the hints, tagged hints and similar example, alone and in every combination, with and without the starter: up to 2,561 programs per lesson, none passes. The same again with the hints' "other names" swapped for the challenge's names. A control shows the audit catches a hint that spells out the answer
Every rule has a message and a sample	`test_every_case_rule_has_a_wrong_sample_and_a_message`
Hidden expected output never sent	The plans, the catalog and lessons, and every check answer for every sample, signed out (backend); every answer the page received (browser)
End to end with real Pyodide, each lesson	Pass, needs_work (with the case's tagged hint first), a Python error with the friendly explanation, line and Python's message (NameError, TypeError, ValueError), an endless loop stopped at 3 seconds, an unsupported import ("Not checked"), every card including a wrong then right prediction and a "Try it" run. Each test checks the page posted exactly the recorded request

Required test	Where
XP and badges once	`test_each_lesson_awards_100_xp_and_its_badge_once`: failing first gives nothing; passing each lesson twice gives 300 XP and 3 badges
Unit 1 unchanged	`unit1-unchanged.spec.ts` (55 texts) passes unchanged; `/api/lessons` equals the frozen fixture; no unit 1 file is in the diff
axe at desktop and phone	The no-mock run above, plus the phone and reduced-motion browser test

Mocks: the browser tests mock the API with recorded **real** answers (`test_the_recorded_answers_are_the_real_apis` regenerates and compares them). Python, the worker, the runner and the page are real. The no-mock run used nothing fake.

Deliberate-break controls (each undone; files matched saved copies byte for byte afterwards)

What I broke	Tests that failed
B1. Lesson 1's approach hint gives the answer ("print(coins // friends) and then print(coins % friends)")	**4**: the hint audit, the renamed-hint audit, "the audit finds the code in the hints", and the recordings
B2. A check that accepts hard-coded output: every lesson 1 case uses the same chest	**7**: hard-coding and visible-only samples now pass, the hard-coded-text test, the hidden-output test, the recordings
B3. The check plan sends hidden cases' expected output to the browser	**7**: ENG-03's two leak tests, the hidden-output test for each of the three lessons, and both recordings tests
B4. The old "Unit complete." ending	**1**: the "after the last released lesson" browser test

7. Known limitations

- **The home page after these three lessons** says "Python Foundations unit 3 is next on your path and is coming soon." The recommendation rules cannot tell a partly released unit from a finished one. It is tested as it is, and listed as question 1.
- **A reply like** `ten` is not a test case (assumption 2).
- **Subtracting a number from a reply that is still text** (`"10" - 3`) gets the general "Python stopped on line 4" explanation; the tagged TypeError hint says to use `int(...)`. A specific translation would be a feedback-engine addition.
- **Practice evidence only**: browser-reported results can be edited by a determined learner (`browser-checks.md` section 5).

- **Not tested with a real screen reader**, or on Safari or Firefox. Reading level was written for grade 9 but not measured with a formula.

8. Live checks for you (after merging and deploying)

You have one test account; your sister can use `/preview` with no account, which saves nothing.

- [ ] **The catalog.** `/courses`: Python Foundations says "2 of 12 units open". `/courses/py`: unit 2 shows three lessons with the summary "…Comparisons, Boolean logic and elif come next."
- [ ] **A full lesson.** Open "Count the treasure" and go through all five cards. On card 3, a wrong answer explains and keeps Continue disabled. On card 4, run the example: "Ran · not checked".
- [ ] **A wrong answer.** On card 5, add `print(coins / friends)` and `print(coins % friends)` and run. "Not yet. For 17 coins and 5 friends, line 1 should be "3", but your program printed "3.4"." The hint button says "Show a hint about this check".
- [ ] **A pass.** Change `/` to `//` and run: "Check passed…", the Treasure counter badge, and "Other ways to do it".
- [ ] **An empty reply.** In "Score the riddle", use `points = int(input())` and run: "Python stopped", "That text is not a whole number", Line 2, and the hint about `""`.
- [ ] **XP.** As your test student, pass all three: 300 more XP and three new badges on the home page; passing again adds nothing.
- [ ] **The ending.** After "Score the riddle": "You finished every lesson released so far…", and the button says "Revisit this quest".
- [ ] **Unit 1 unchanged.** "Open the story" looks and behaves as before.
- [ ] **For your sister:** is anything confusing or too long for a grade 9 reader? Card 1 of "Average the scores" is the densest.
- [ ] **No CSP reports** in DevTools → Console.

9. Questions for the owner

1. **The home card** after the three lessons says unit 3 is next. Accept it until part 2, or add a way to mark a unit as "more lessons coming" (a content field and an additive API field) in part 2?
2. **Assumption 1:** lessons 1 and 2 hand over the `int(input())` lines before lesson 3 explains them. OK?
3. **Assumption 2:** an empty reply is tested; `ten` is taught but not tested. OK, or do you want an "expected error" case type later?
4. **Assumption 3:** fix-the-bug done through the predict and "Try it" cards. Do you want a real fix-the-bug card type (a schema change) before part 2?
5. **Construct rules** (none were needed): if you want them for part 2 (for example to require `elif` in `py.elif`), they need names that fit an ID, like `uses-elif` or `no-eval`; the judge to return `<lesson>.require.<name>` or `<lesson>.forbid.<name>`; and `content.feedback_errors` to accept those tags (handoff 22 section 7).

6. **Badges:** one per lesson, as you asked, while `course-catalog.md` 6.2 says no per-lesson badges for new lessons. Which should the plan say?
7. **Wording:** anything in the appendix you want changed? A change is a content edit plus a version bump, and the tests re-check it.

10. Proposed next PR (EDU-04 part 2)

`py.compare` (comparison operators on numbers), `py.boolean-logic` (`and`, `or`, `not`) and `py.elif`, continuing the riddle: is the answer right, too high or too low. Then the unit summary and the ending can say the unit is complete, and question 1 goes away.

11. For your docs PR (not edited here)

Where	What
Backlog EDU-04	proposed → in_review for part 1 (this PR, handoff 24); part 2 proposed
Stage table	"24. EDU-04 part 1 | Unit 2 lessons 1 to 3 | in_review (handoff 24)"
`course-catalog.md` 6.2	The per-lesson badge line (question 6)
Decision log	A chapter from sections 4 to 6 of this handoff

12. Removing the worktree safely

From the **main** folder, after this PR is merged:

```
cd /Users/DEV/Projects/python-academy
git -C ../python-academy-u02 status --short   # expect nothing, or only ??
frontend/AGENTS.md
rm -f ../python-academy-u02/frontend/AGENTS.md   # generated by Next.js, not part of the
project
git worktree remove ../python-academy-u02     # deletes only that folder; refuses if
anything is uncommitted
git worktree prune
git branch -d content/py-u02-part1             # refuses if the branch is not merged
```

Don't delete the folder in the Finder or with `rm -rf`; Git would keep a stale record of it (`git worktree prune` fixes that).

Appendix. Every card, hint and check case, word for word

Generated from the three content files. Hidden cases' expected output is listed here for review only; it never reaches the browser.

A.1 `py.arithmetic`: Count the treasure

Concept: Arithmetic · + - * / // %. Description: Give your adventure a score, and share out a chest of coins. Badge: **Treasure counter** (`py.badge.treasure-counter`, practice).

Card	Text
1. Discover: Python can do the maths	Numbers in Python have no quotes around them. + adds, - takes away, * multiplies and / divides. print(3 + 4) shows 7, and a variable can hold a number, like score = 10. Two more operators help when you share things out. // divides and keeps only the whole part: 17 // 5 is 3. % gives what is left over: 17 % 5 is 2. As in maths class, Python works out * and / before + and -. Use parentheses to choose a different order.
2. Worked example	`# Work out the hero's score` / `gems = 4` / `coins = 13` / `score = gems * 10 + coins` / `print(score)` / `# Share the coins between 5 friends` / `print(coins // 5)` / `print(coins % 5)` Replies: none. Output: `53` / `2` / `3` Python works out gems * 10 first, so the score is 40 + 13. Then // gives each friend's whole coins, and % gives the coins left over.
3. Predict	If coins were 14 instead of 13, what would print(coins % 5) show? Options: **4** (answer); 2; 2.8 Why: Five friends take 2 coins each, which uses 10 of the 14. % shows what is left over, so it prints 4. 14 // 5 would show 2, and 14 / 5 would show 2.8.
4. Try it	Give gems and coins new values. Before you run, predict all three lines, then run and compare. Try a number of coins that shares out exactly between 5, like 15: what does % show then?
5. Challenge	The party opens a chest of coins to share. The starter asks for the number of coins and the number of friends. Print two lines: how many whole coins each friend gets, then how many coins are left over. Print just the two numbers. The check runs your program with several chests, including some you can't see. Starter: `# These two lines ask for whole numbers (lesson 3 explains how)` / `coins = int(input("How many coins? "))` / `friends = int(input("How many friends? "))` / `# Print each friend's whole share, then the coins left over`

Hint	Text
A nudge	You need two results from the same two numbers: the whole coins each friend gets, and the coins left over.
A bigger hint	Look at the comment line. One operator divides and keeps only the whole part; another gives what is left over. Each result goes in its own print().
The approach	A pattern with other names: print(cookies // children) shows each child's whole cookies. For the leftover line, use the operator that gives what is left over.

Hint	Text
About `NameError`	Use the names the starter made, coins and friends, spelled exactly the same in your print() lines.
About `TypeError`	Print each number on its own, like print(coins). Joining a number to text with + needs str(), and this check wants only the numbers.
About `py.arithmetic.case.even`	Compare your first line with 3. A decimal point, like 3.4, means / was used: // keeps only the whole coins. Words on the line? Print just the number.
About `py.arithmetic.case.fewer`	With 3 coins and 5 friends, nobody gets a whole coin, so the first line is 0 and all 3 coins are left over. A division that rounds up would give 1 instead.
About `py.arithmetic.case.big`	Your program needs to work for any chest. Use coins and friends in your calculations, not numbers typed into the code.
Similar example	`start = 9` / `later = start + 14` / `print(later % 12)` 14 hours after 9 o'clock, a 12-hour clock shows 11: % keeps what is left after taking away whole groups of 12. Your challenge makes whole groups too, one share for each friend.
Other way, after passing	`coins = int(input("How many coins? "))` / `friends = int(input("How many friends? "))` / `share, left = divmod(coins, friends)` / `print(share)` / `print(left)` divmod() works out the whole share and the leftover in one step.
Other way, after passing	`coins = int(input("How many coins? "))` / `friends = int(input("How many friends? "))` / `share = coins // friends` / `print(share)` / `print(coins - share * friends)` The leftover is what remains after every friend takes their share.

Case	Visibility	Description (shown to learners)	Replies	Expected output
`even`	visible	17 coins and 5 friends	'17', '5'	`3` / `2`
`exact`	visible	20 coins and 4 friends	'20', '4'	`5` / `0`
`fewer`	visible	3 coins and 5 friends	'3', '5'	`0` / `3`
`big`	hidden	a big chest	'1000', '7'	`142` / `6`
`one-friend`	hidden	a chest for one friend	'9', '1'	`9` / `0`

A.2 `py.int-float`: Average the scores

Concept: Numbers · int and float. Description: Find the average of two riddle scores, decimals and all. Badge: **Score keeper** (`py.badge.score-keeper`, practice).

Card	Text
1. Discover: Two kinds of numbers	Python has two kinds of numbers. An int is a whole number, like 7 or -3. A float has a decimal point, like 2.5 or 7.0. Dividing with / always gives a float, even when the answer is whole: 6 / 3 is 2.0. When a calculation mixes an int and a float, the answer is a float: 2 + 0.5 is 2.5. int() cuts off the decimal part without rounding, so int(3.9) is 3 and int(-2.5) is -2. A few decimals print oddly, like 0.1 + 0.2 showing 0.30000000000000004. That comes from how computers store decimals, not from a mistake in your code.
2. Worked example	`# How long the walk to the market takes` / `distance = 9` / `speed = 2` / `hours = distance / speed` / `print(hours)` / `# The whole hours, with the decimal part cut off` / `print(int(hours))` / `# / gives a float even when the answer is whole` / `print(8 / 2)` Replies: none. Output: `4.5` / `4` / `4.0` hours holds the float 4.5. int() cuts off the .5 and gives the int 4. The last line shows 4.0, not 4, because / always gives a float.
3. Predict	What would print(int(7.9)) display? Options: 8; **7** (answer); 7.9 Why: int() cuts off everything after the decimal point; it does not round up. So int(7.9) is 7, even though 7.9 is close to 8.
4. Try it	Change distance to 10. Before you run, predict the first line: will it still have a decimal point? Then change the last line to print(8 // 2), predict again, and compare the two answers.
5. Challenge	After two riddles, the hero has two scores. A wrong answer can make a score negative. The starter asks for both scores. Print two lines: first the average of the two scores, then the same average with its decimal part cut off by int(). Print just the two numbers. Starter: `# These two lines ask for whole-number scores (lesson 3 explains how)` / `first = int(input("First score? "))` / `second = int(input("Second score? "))` / `# Work out the average, then print it and its whole part`

Hint	Text
A nudge	The average of two scores is their total divided by 2. You print it twice: once as it is, and once with its decimal part cut off.
A bigger hint	Look at the order Python works in: it divides before it adds. Check that your whole total gets divided, not only the second score.
The approach	With other names: total = low + high adds first, and then total / 2 halves it. For your second line, think about which function cuts off the decimal part.
About `NameError`	Use the names from the starter, first and second, and give your average a name before you print it.
About `TypeError`	Print each number on its own, like print(total). Joining a number to text with + needs str(), and this check wants only the numbers.

Hint	Text
About `py.int-float.case.half`	The average of 7 and 8 is 7.5. If your first line says 11.0, Python divided only the second score: put the addition inside parentheses.
About `py.int-float.case.negative`	With -3 and 2, the average is -0.5. int() cuts off the decimal part, so the second line is 0. The // operator rounds down instead, which gives -1 for negative numbers.
About `py.int-float.case.high`	Your program needs to work for any two scores. Use first and second in the calculation, not numbers typed into the code.
Similar example	`morning = 4` / `evening = 9` / `middle = (morning + evening) / 2` / `print(middle)` Halfway between a morning temperature of 4 and an evening of 9 is 6.5. Your challenge reads its two numbers from the replies, and also prints the whole part.
Other way, after passing	`first = int(input("First score? "))` / `second = int(input("Second score? "))` / `total = first + second` / `print(total / 2)` / `print(int(total / 2))` Keep the total in its own variable first.
Other way, after passing	`first = int(input("First score? "))` / `second = int(input("Second score? "))` / `average = (first + second) * 0.5` / `print(average)` / `print(int(average))` Multiplying by 0.5 halves a number too, and the answer is also a float.

Case	Visibility	Description (shown to learners)	Replies	Expected output
`half`	visible	scores of 7 and 8	'7', '8'	`7.5` / `7`
`whole`	visible	scores of 6 and 8	'6', '8'	`7.0` / `7`
`negative`	visible	scores of -3 and 2	'-3', '2'	`-0.5` / `0`
`high`	hidden	two high scores	'95', '100'	`97.5` / `97`
`equal`	hidden	two equal scores	'10', '10'	`10.0` / `10`

A.3 `py.convert-input`: Score the riddle

Concept: Conversion · int(input()). Description: Turn the player's replies into numbers, and keep score when a riddle is solved or missed. Badge: **Riddle scorer** (`py.badge.riddle-scorer`, practice).

Card	Text
1. Discover: Replies are always text	input() always gives back text, even when the player types digits. So "5" + "5" joins into "55" instead of adding up to 10. int(...) turns text like "12" into the whole number 12, and float(...) turns "2.5" into the decimal 2.5. Wrap the reply in int(), like points = int(input("Points? ")). If the reply is not a whole number, such as ten or an empty reply, int() stops with a ValueError. When a reply might be empty, compare it with "" before you convert it, as the example on the next card does.
2. Worked example	`# Ask for the gold in your pocket, as a whole number` / `gold = int(input("Gold in your pocket? "))` / `# Ask for the bridge toll; an empty reply means no toll` / `toll = input("Toll? ")` / `if toll == "":` / `toll = "0"` / `# Turn the toll into a number, then pay it` / `print(gold - int(toll))` Replies: '20', '5'. Output: `15` With 20 and 5 as replies, it prints 15. An empty toll becomes the text "0" first, so int() has a digit to read. Without int(), Python could not take text away from a number.
3. Predict	If the player types ten in letters for the gold, what happens? Options: It prints 5; **Python stops with a ValueError** (answer); It prints ten Why: int() only understands digits, like "10". The letters ten are not a whole number, so Python stops with a ValueError on line 2. You can try it on the next card.
4. Try it	Run the example with 20 and 5. Then predict, and run, each of these: an empty toll, and the letters ten as the gold. Last, delete int(and its) from the gold line, run it, and read the explanation. Reset code puts the example back.
5. Challenge	The riddle keeper says how many points a riddle is worth. Ask for the hero's score, then for the riddle's points, and turn both replies into whole numbers yourself. Print two lines: the score if the riddle is solved (the points are added), then the score if it is missed (the points are taken away). An empty points reply means a practice riddle worth 0. Print just the two numbers. Starter: `# Ask for the score and turn the reply into a whole number` / `# Ask for the riddle's points; an empty reply means 0` / `# Print the score if solved, then the score if missed`

Hint	Text
A nudge	Three steps: get the score as a number, get the points as a number (or 0 when the reply is empty), then print the two results.
A bigger hint	Look at the second comment. Keep the points reply as text until you have checked whether it is "", and only then turn it into a number.
The approach	A pattern with other names: reply = input("Coins? "), then if reply == "": with an indented line that gives reply the text "0", then coins = int(reply).
About `ValueError`	One test leaves the points reply empty, and int() can't turn empty text into a number. Check whether the reply is "" before you call int().
About `TypeError`	input() gives text. Text can't be taken away from a number, and + joins text instead of adding. Wrap each reply in int(...) before you calculate.

Hint	Text
About `NameError`	Give each name a value before you use it, spelled the same each time. Each input() reply needs a name of its own.
About `EOFError`	This challenge gives your program two replies: the score, then the points. Use exactly two input() calls.
About `py.convert-input.case.solved-missed`	With a score of 10 and a riddle worth 3, the first line is 13 (solved) and the second is 7 (missed). Check the order of your two print() lines.
About `py.convert-input.case.below-zero`	Missing a riddle can take the score below zero: 5 - 8 is -3. Print the result as it is; a negative score is fine here.
About `py.convert-input.case.no-points`	With an empty points reply, the riddle is worth 0, so both lines show the score unchanged.
About `py.convert-input.case.large`	Calculate from the replies, not from numbers typed into the code, so your program works for every score.
About `py.convert-input.case.from-zero`	Every reply counts, even a score of zero. Calculate from the replies rather than checking for particular numbers.
Similar example	`reply = input("Lanterns? ")` / `if reply == "":` / `reply = "1"` / `lanterns = int(reply)` / `print(lanterns * 2)` An empty reply becomes the text "1" before int() reads it, so the program never stops. Your challenge uses 0 for an empty reply, and needs two numbers.
Other way, after passing	`score = int(input("Score? "))` / `points = input("Points? ")` / `if points == "":` / `points = 0` / `else:` / `points = int(points)` / `print(score + points)` / `print(score - points)` if and else can each give points its number.
Other way, after passing	`score = int(input("Score? "))` / `points = int(input("Points? ") or "0")` / `print(score + points)` / `print(score - points)` or picks "0" when the reply is empty. A later lesson explains or properly.

Case	Visibility	Description (shown to learners)	Replies	Expected output
`solved-missed`	visible	a score of 10 and a riddle worth 3	'10', '3'	`13` / `7`
`below-zero`	visible	a score of 5 and a riddle worth 8	'5', '8'	`13` / `-3`
`no-points`	visible	a score of 6 and an empty points reply	'6', ''	`6` / `6`
`large`	hidden	a large score	'250', '120'	`370` / `130`
`from-zero`	hidden	starting from a score of zero	'0', '4'	`4` / `-4`